

# Bài Tập Cuối Chương

1. Tình huống nào sau đây mô tả vấn đề "Props Drilling" trong React?

- A. Khi một component con không nhận được props cần thiết từ component cha.
  - B. Khi dữ liệu props bị thay đổi ngoài ý muốn trong quá trình truyền xuống các component con.
  - C. Khi props phải được truyền qua nhiều tầng component trung gian mà các component trung gian này không thực sự sử dụng props đó.
  - D. Khi props được truyền lên component cha từ component con.
2. Mục đích chính của Context API trong React là gì?

- A. Thay thế hoàn toàn cho việc sử dụng props trong React.
  - B. Quản lý state toàn cục cho toàn bộ ứng dụng, tương tự như Redux.
  - C. Giải quyết vấn đề Props Drilling bằng cách cho phép chia sẻ dữ liệu dễ dàng hơn giữa các component con cháu trong một phạm vi nhất định.
  - D. Tối ưu hóa hiệu suất render của ứng dụng React bằng cách giảm số lần re-render.
3. Context API **KHÔNG** phù hợp để giải quyết vấn đề nào sau đây?

- A. Chia sẻ theme (chế độ sáng/tối) cho toàn bộ ứng dụng.
  - B. Chia sẻ thông tin người dùng hiện tại cho các component khác nhau trong ứng dụng.
  - C. Quản lý state ứng dụng phức tạp, cần undo/redo, logging, và các tính năng nâng cao khác.
  - D. Chia sẻ ngôn ngữ ứng dụng (đa ngôn ngữ) cho các component hiển thị văn bản.
4. Ưu điểm chính của việc sử dụng Context API so với Props Drilling là gì?

- A. Code chạy nhanh hơn và ứng dụng có hiệu suất cao hơn.
  - B. Code dễ đọc, dễ bảo trì và quản lý hơn khi chia sẻ dữ liệu qua nhiều tầng component.
  - C. Context API có thể thay thế hoàn toàn cho việc sử dụng state trong React.
  - D. Context API giúp giảm dung lượng bundle của ứng dụng React.
5. Chọn phát biểu **SAI** về Context API:

- A. Context API giúp giải quyết vấn đề Props Drilling.
  - B. Context API cho phép chia sẻ dữ liệu "toàn cục" trong một phạm vi nhất định.
  - C. Context API là một thư viện quản lý state toàn cục mạnh mẽ như Redux.
  - D. Context API bao gồm các thành phần chính như Provider và Consumer.
6. Để tạo ra một Context trong React, chúng ta sử dụng hàm nào?

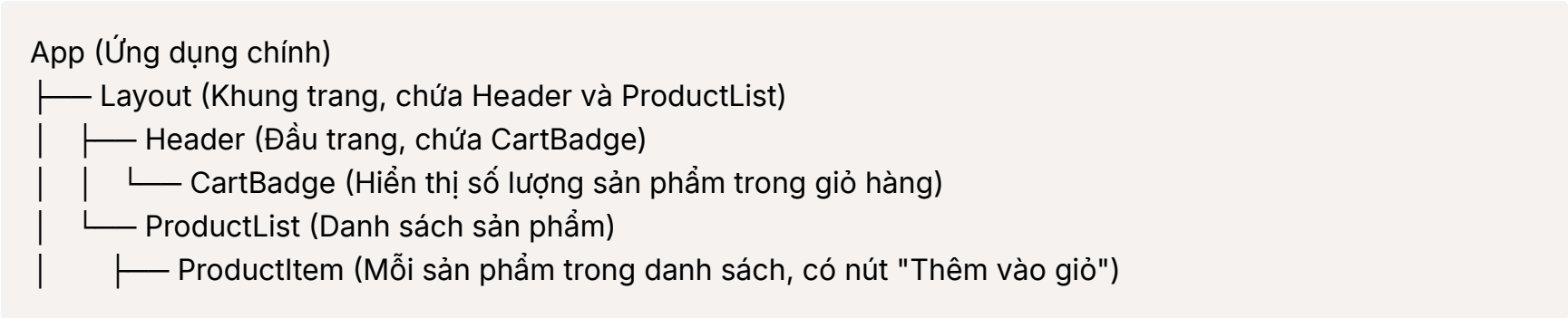
- A. createContext.Provider()
  - B. Context.Provider()
  - C. createContext()
  - D. useContext()
7. Component MyContext.Provider cần có thuộc tính nào **bắt buộc** để cung cấp dữ liệu Context cho các component con cháu?

- A. children
  - B. context
  - C. valueContext
  - D. value
8. Điều gì xảy ra khi giá trị value của MyContext.Provider thay đổi?

- A. Chỉ component Provider re-render.
- B. Chỉ các component Consumer re-render.
- C. Chỉ các component useContext re-render.
- D. Tất cả các component Consumer và useContext bên dưới Provider sẽ re-render.

9. Bạn hãy xây dựng ứng dụng React hiển thị danh sách sản phẩm và "Cart Badge" (biểu tượng giỏ hàng) trên Header, hiển thị số lượng sản phẩm trong giỏ.

Cấu trúc component của ứng dụng sẽ như sau (**Lưu ý:** đây là cấu trúc component, không phải là cấu trúc folder):



Hãy viết chương trình thực hiện các yêu cầu sau (có thể đặt **tất cả các file này vào trong thư mục `src/`** của dự án)

- **Tạo file `CartContext.js` :**
  - Trong file này, bạn hãy **tạo ra một Context** cho giỏ hàng.
  - Sử dụng hàm `createContext()` từ React để tạo Context.
  - Đặt **giá trị ban đầu** cho Context như sau: một **object** có hai thông tin:
    - `cartItemCount` : **Số lượng sản phẩm trong giỏ, bắt đầu là 0.**
    - `setCartItemCount` : **Một function trống**, bạn chưa cần viết code gì cho function này vội.
- **Tạo `CartProvider` component trong file `App.js` :**
  - Mở file `App.js` của bạn.
  - Viết một component mới tên là `CartProvider` . Đây sẽ là component cung cấp dữ liệu giỏ hàng cho ứng dụng.
  - Trong `CartProvider` , bạn cần **quản lý state** cho số lượng sản phẩm trong giỏ hàng. Hãy sử dụng Hook `useState` để tạo một state tên là `cartItemCount` , và **bắt đầu với giá trị 0**. Bạn cũng sẽ có một function tên là `setCartItemCount` để thay đổi giá trị của `cartItemCount` .
  - Trong phần code hiển thị (render) của `CartProvider` , bạn cần sử dụng `CartContext.Provider` . Đây là cách để **"chia sẻ"** dữ liệu Context.
  - **Quan trọng:** Khi sử dụng `CartContext.Provider` , bạn cần **truyền vào một prop là `value`** . Giá trị của prop `value` này sẽ là một **object** chứa:

```
{  
  cartItemCount: ← Giá trị state `cartItemCount` mà bạn đã tạo ở trên  
  setCartItemCount: ← Function `setCartItemCount` mà bạn đã tạo ở trên  
}
```
  - Để `CartProvider` có thể bao bọc và cung cấp Context cho các phần khác của ứng dụng, hãy chắc chắn rằng bên trong `CartContext.Provider` , bạn **hiển thị `children`** .
  - **Cuối cùng, trong file `App.js` , hãy đảm bảo rằng component `Layout` được "bọc" bên trong component `CartProvider` .** Điều này có nghĩa là `<CartProvider>` sẽ bao quanh `<Layout>` .
- **Hiển thị số lượng giỏ hàng trong `CartBadge` component (file `CartBadge.js` ):**
  - Mở file `CartBadge.js` .
  - Component này sẽ **hiển thị số lượng sản phẩm trong giỏ hàng**.
  - Để lấy được số lượng giỏ hàng từ Context, bạn cần sử dụng Hook `useContext(CartContext)` . Hook này sẽ **"lấy ra"** giá trị mà `CartProvider` đã cung cấp cho `CartContext` .
  - Sau khi dùng `useContext` , bạn sẽ có được object giá trị Context, trong đó có `cartItemCount` .

- Hãy **hiển thị** số lượng này. Ví dụ, bạn có thể dùng code sau trong phần hiển thị của `CartBadge` : `<div>Giỏ hàng {cartItemCount}</div>` .

- **Xử lý nút "Thêm vào giỏ" trong `ProductItem` component (file `ProductItem.js` ):**

- Mở file `ProductItem.js` .
- Trong component này, bạn sẽ thấy nút "Thêm vào giỏ".
- Khi người dùng **nhấn vào nút "Thêm vào giỏ"**, bạn muốn **tăng số lượng sản phẩm trong giỏ hàng lên 1**.
- Để làm được điều này, trong `ProductItem` , bạn cũng cần dùng Hook `useContext(CartContext)` để **"lấy ra"** giá trị Context.
- Từ giá trị Context, bạn sẽ có được function `setCartItemCount` .
- Trong function được gọi khi nút "Thêm vào giỏ" được click (event `onClick` ), hãy **gọi function `setCartItemCount` để tăng giá trị `cartItemCount` lên 1**. (Gợi ý: Bạn có thể dùng `setCartItemCount(cartItemCount + 1)` hoặc `setCartItemCount(oldValue ⇒ oldValue + 1)` ).

- **Viết code cho các component còn lại ( `App.js` , `Layout.js` , `Header.js` , `ProductList.js` ):**

- Trong các file `App.js` , `Layout.js` , `Header.js` , `ProductList.js` , bạn hãy viết code đơn giản để **tạo ra cấu trúc ứng dụng** như đã mô tả ở trên (App → Layout → Header, ProductList → ProductItem).
- Các component này không cần có logic gì phức tạp, chỉ cần đảm bảo:
  - `App` component hiển thị `CartProvider` và bên trong là `Layout` .
  - `Layout` component hiển thị `Header` và `ProductList` .
  - `Header` component hiển thị `CartBadge` .
  - `ProductList` component hiển thị **danh sách các `ProductItem`** .
  - Để có danh sách sản phẩm, bạn có thể tạo một **mảng dữ liệu sản phẩm đơn giản** (ví dụ, một mảng các object, mỗi object mô tả một sản phẩm với tên và giá) ngay trong file `ProductList.js` . Không cần lấy dữ liệu từ bên ngoài.

- Hãy chắc chắn rằng ứng dụng của bạn **chạy đúng**: Khi bạn nhấn nút "Thêm vào giỏ" ở mỗi sản phẩm, thì **số lượng hiển thị trên "Cart Badge" ở Header phải tăng lên** đúng số lượng sản phẩm bạn đã thêm vào.

10. Viết chương trình đồng hồ bấm giờ đơn giản theo các yêu cầu sau:

- Tạo một component tên là Stopwatch.
- Hiển thị thời gian hiện tại của đồng hồ bấm giờ (ban đầu là 00:00:00). Định dạng thời gian: HH:MM:SS (giờ:phút:giây).
- Thêm các nút bấm: "Start", "Stop", "Reset".
- **Chức năng:**
  - **"Start"**: Bắt đầu đếm thời gian từ thời điểm bấm nút. Thời gian trên đồng hồ liên tục cập nhật mỗi giây.
  - **"Stop"**: Dừng đếm thời gian. Thời gian hiển thị giữ nguyên tại thời điểm bấm nút.
  - **"Reset"**: Dừng đếm thời gian (nếu đang chạy) và đặt lại thời gian hiển thị về 00:00:00.
- **Sử dụng `useRef` để:**
  - Lưu trữ ID của timer ( `setInterval` ) khi đồng hồ đang chạy. Điều này giúp bạn có thể xóa interval khi "Stop" hoặc "Reset".
- **Sử dụng `useState` để:**
  - Quản lý state thời gian hiển thị trên đồng hồ (ví dụ: state object `{ hours, minutes, seconds }` ).
  - Quản lý trạng thái đồng hồ (ví dụ: `isRunning: boolean` ) để biết đồng hồ đang chạy hay dừng (có thể dùng để điều khiển trạng thái nút "Start"/"Stop").

11. Ứng dụng Đo kích thước Element (Element Size Measurement)

Viết chương trình theo các yêu cầu sau:

- Tạo một component tên là ElementSizeMeasurer.

- Hiển thị một `<div>` có một đoạn văn bản bên trong. Đoạn văn bản có thể là bất kỳ nội dung gì (ví dụ: "Đây là một đoạn văn bản ví dụ. Kích thước của khung chứa này sẽ được đo.").
- Thêm một nút bấm có nội dung "Đo kích thước".
- Khi nút "Đo kích thước" được click:
  - Sử dụng `useRef` để tạo một ref và gán ref này cho thẻ `<div>` chứa văn bản.
  - Sau khi nút được click, lấy DOM element tương ứng với ref này (thông qua `ref.current`).
  - Sử dụng các thuộc tính DOM như `offsetWidth` và `offsetHeight` để lấy chiều rộng và chiều cao của DOM element.
  - Hiển thị chiều rộng và chiều cao đo được ngay bên dưới nút bấm (ví dụ: "Chiều rộng: [width]px, Chiều cao: [height]px").
  - Để hiển thị kích thước đo được lên UI, bạn cần sử dụng `useState` để quản lý state hiển thị kích thước.
- **Quan sát:**
  - Khi bạn click nút "Đo kích thước", kích thước của `<div>` có được hiển thị đúng không?
  - Điều gì xảy ra nếu bạn thay đổi nội dung văn bản bên trong `<div>` (ví dụ, thêm chữ, xóa chữ) và click "Đo kích thước" lại? Kích thước hiển thị có cập nhật theo nội dung mới không?
  - Component có re-render mỗi khi bạn click nút "Đo kích thước" không? (Gợi ý: Thêm `console.log('Render ElementSizeMeasurer')` để theo dõi re-render).